



CENTRO UNIVERSITÁRIO DE BRASÍLIA
FACULDADE DE TECNOLOGIA E CIÊNCIAS SOCIAIS APLICADAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO
PROJETO INTEGRADOR I — TURMA C

Professora Orientadora: Kadidja Valéria Reginaldo de Oliveira

Arquitetura e Funcionamento do Sistema MAPPA

Victor Fernandes 22407498

João Vitor Velozo 22403388

Danilo Lisboa 22401625

Igor Cavalcante 22401315

Lucas Monteiro 22503356

BRASÍLIA, junho de 2026

1. INTRODUÇÃO

A arquitetura da solução foi dividida em duas etapas principais, garantindo o desacoplamento entre o processamento dos dados e a sua visualização:

ETL: Script em Python responsável por extrair, limpar e preparar os dados brutos.

Dashboard Interativo: Aplicação web foi construída com a biblioteca Streamlit para consumo e exploração visual dos dados processados.

2. Processo de ETL

O arquivo `etl_mappa_v3.py` atua como o motor de processamento do projeto. Ele foi projetado para ler múltiplos arquivos CSV brutos disponibilizados pela PRF e transformá-los em uma base analítica otimizada.

O funcionamento do ETL é estruturado nas seguintes fases:

- **Extract:** O script varre um diretório de entrada (`dados_prf`) em busca de arquivos CSV. Ele possui inteligência para detectar automaticamente o tipo de separador (vírgula ou ponto e vírgula) e o encoding (latin1 ou utf-8), garantindo que bases de diferentes anos sejam lidas corretamente.
- **Transform:** Esta é a etapa mais densa do código, responsável por garantir a qualidade dos dados:
- **Padronização de Colunas:** Normaliza os nomes das colunas, removendo acentos e caracteres especiais, e unifica nomenclaturas de bases antigas e novas (ex: `id_ocorrendia` vira `id`, `brasil` vira `br`).
- **Limpeza e Tipagem:** Converte campos numéricos e trata strings nulas ou vazias. Há uma função robusta (`parse_data_robusta`) dedicada exclusivamente a entender múltiplos formatos de data e unificá-los.
- **Derivação de Variáveis:** O script cria novas colunas que serão essenciais para o painel, como a `faixa_horaria` (agrupando horários em blocos como "06-11" ou "12-17"), e o `nivel_severidade` (classificando o acidente como "SEM VITIMAS", "COM FERIDOS" ou "COM MORTOS").
- **Recorte do Projeto:** Filtra a base monumental do Brasil inteiro para focar estritamente no escopo do projeto: as UFs (DF, GO, MG) e as rodovias específicas (BRs 020, 040, 060, 070 e 080)
- **Carga (Load):** Em vez de jogar tudo em um banco de dados complexo, o script adota uma abordagem de arquivos estáticos, exportando tabelas Fato (ocorrências e mapa), tabelas Dimensão (causas, tipos, municípios)

e agregações pré-calculadas (por mês, por BR, por faixa horária) para a pasta `saida_dashboard`. Isso garante que o painel Streamlit seja extremamente rápido, pois não precisa realizar cálculos pesados em tempo real.

3. Dashboard Interativo

O arquivo `app_mappa_streamlit_v2.py` é a interface de usuário final. Ele consome os arquivos CSV limpos gerados pelo ETL e os transforma em um painel interativo. O uso do Streamlit permite que o código Python seja renderizado diretamente como uma página web.

O fluxo de funcionamento do Dashboard é organizado da seguinte forma:

- **Carregamento e Cache:** O aplicativo utiliza o decorador (`@st.cache_data`) do Streamlit ao ler a pasta (`saida_dashboard`). Isso significa que os CSVs são carregados na memória do servidor apenas na primeira vez que o painel é aberto, melhorando drasticamente a performance para o usuário.
- **Barra Lateral de Filtros:** O menu lateral permite que o usuário aplique filtros dinâmicos por Ano, Rodovia, Município, Severidade e Causa. Sempre que um filtro é alterado, a função (`build_filtered`) recalcula o recorte e a função (`agg_filtered`) refaz as agregações matemáticas dinamicamente.
- **Estrutura de Abas:** O conteúdo é dividido em seis visões temáticas para guiar a jornada do usuário:
- **Visão Geral:** Apresenta os KPIs no topo, como total de acidentes, feridos e mortos, seguidos por gráficos de barras resumidos
- **Análise Exploratória de Dados (AED):** Exibe a estrutura da base filtrada, quantificando valores nulos e mostrando gráficos de pizza e barras horizontais para severidade e tipos de acidentes.
- **Dashboard:** O núcleo analítico, focado em cruzamentos visuais. Utiliza a biblioteca `plotly` para gerar gráficos de séries temporais e rankings interativos
- **Mapa:** Exclusivo para dados georreferenciados. A aplicação limpa coordenadas inválidas e plota um mapa interativo utilizando (`px.scatter_map`), colorindo os pontos pela severidade do acidente. Inclui uma limitação técnica de amostrar até 8.000 pontos para não travar o navegador do usuário.
- **Insights:** Um módulo algorítmico simples (`generate_insights`) que lê as agregações atuais do painel e gera frases de texto automáticas, facilitando a interpretação imediata dos dados.

- **Sobre a Base:** Funciona como um dicionário de dados e documentação interna, explicando as limitações da base e o significado de cada variável.

4. Execução Integrada

Para que o ecossistema funcione adequadamente, o fluxo operacional obedece a uma ordem estrita:

- Os dados brutos da PRF devem ser alocados na pasta de entrada.
- O analista executa o script `etl_mappa_v3.py`, que atua de forma autônoma (sem necessidade de banco de dados ou autenticação).
- Após a geração da pasta `saida_dashboard`, as dependências (pandas, plotly, streamlit, numpy) devem estar ativas no ambiente
- O servidor web é iniciado via linha de comando (`streamlit run app_mappa_streamlit_v2.py`), expondo a interface gráfica no navegador local.